



St Mary's
University
Twickenham
London

KNIGHTS OF ELDORIA

Author : 2317527
Course Title : MSc Computer Science
Module Convenor : Prins Butt
Date : 27th March 2025

Table of Contents

KNIGHTS OF ELDORIA 1

1 Overview.....4

1.1 Aim & Objectives4

1.2 System Design & Architecture.....4

1.3 Scope & Limitations5

2 Modules / Functionality6

2.1 Process Logic (AI & Gameplay).....7

2.2 Visual Implementation (Pygame 2D Grid).....7

2.3 Main Game Module (Code Breakdown).....8

3 Recommendations12

3.1 Future Enhancements12

4 References14

5 Appendices15

Index of Tables

Table 1Requirements Implementation Status6

Table of Figures

Figure 1Screenshot of Grid Class code.....9

Figure 2 Screenshot of Treasure Class.....9

Figure 3 Screenshot of Treasure Hunter Class..... 10

Figure 4 Screenshot of Knight Class..... 11

Figure 5 Screenshot of Hideout Class..... 11

Figure 6 Screenshot of Simulation Class 12

1 Overview

1.1 Aim & Objectives

The Knights of Eldoria project was conceived as a fantasy-themed simulation system rooted in the concepts of artificial intelligence and real-time strategic decision-making. The aim was not just to create a functioning game, but to simulate a world in which intelligent agents—treasure hunters and knights—interact in a dynamic environment using well-defined behavioural rules and decision models.

The project merges theoretical foundations of AI with practical implementation. It explores how pathfinding algorithms such as A* influence agent navigation, how memory and prediction logic affect adversarial pursuit (in the case of knights), and how cooperative knowledge sharing can be simulated among autonomous units (treasure hunters). These principles are widely applicable in real-world systems, such as logistics, surveillance, and autonomous robotics.

Knights of Eldoria is more than a simple game; it is a controlled testbed where AI behaviour's can be observed, tuned, and enhanced. The entire simulation takes place on a toroidal 2D grid representing the kingdom of Eldoria, with randomized spawning of treasures and hideouts, ensuring unique challenges in every execution. Each simulation step acts as a discrete event model where AI agents perceive, decide, and act based on their programmed logic, constrained by their stamina, memory, and strategic objectives.

As an academic project, it provides an ideal case study for evaluating AI design trade-offs—rule-based logic versus adaptive learning, prediction versus random patrols, stamina thresholds as a resource constraint, and the challenges of balancing performance with game realism. By grounding these ideas in an immersive simulation, the project bridges theory and practice, preparing the foundation for both future research and real-world applications of autonomous systems.

1.2 System Design & Architecture

The system is designed as a modular and object-oriented simulation using Python and the Pygame library. The game world is represented as a 2D grid where each cell can contain various entities, such as hunters, knights, treasures, or hideouts. At the core of the architecture is a Grid class that

manages wrap-around movement and spatial logic. The game logic is divided into functional classes: Hunter, Knight, Treasure, Hideout, and the central Simulation controller.

Each agent (hunter or knight) functions autonomously, operating based on internal logic and environment perception. Hunters seek out treasure using A* pathfinding, monitor their stamina, and coordinate with other hunters. Knights, on the other hand, act as adversaries, patrolling the map and detecting hunters within a defined radius. They use predictive logic to anticipate hunter movement, ensuring the AI remains competitive.

The architecture also includes a sound engine to trigger specific sound effects based on interactions, such as footsteps, collecting treasure, or evading knights. The Simulation class manages frame updates, game termination, and entity updates. This separation of logic enables scalability and future integration of more complex AI or multiplayer features.

1.3 Scope & Limitations

The project scope was defined to model a complete AI-based simulation in a fantasy grid world. The game simulates real-time resource collection, agent survival, and adversarial avoidance within a defined set of rules and capabilities. All agents are autonomous, relying on internal logic and environmental awareness to perform actions. The simulation handles spawning, movement, resource decay, and decision-making without direct user control.

Limitations include the absence of adaptive learning mechanisms such as reinforcement learning. While the current AI is competent, it does not learn from past behaviour. There is also no multiplayer or player-controlled mode. Visuals are minimalist, focusing on gameplay logic over artistic quality. Grid-based movement, although precise, lacks smooth animation transitions that might enhance realism.

Nevertheless, the system serves as a solid base for experimentation with AI strategies, resource prioritization, and team dynamics. The scalability of the architecture means additional complexity—like learning algorithms or improved path prediction—can be integrated.

Table 1 Requirements Implementation Status

Requirement	Description	Status
2D Grid Representation	Grid with wrap-around mechanics	Completed
Treasure Types & Decay	Bronze, Silver, Gold; decay 0.1% per step	Completed
Hunter AI	Pathfinding, stamina, teamwork, memory	Completed
Knight AI	Patrols, memory-based prediction, chases hunters	Completed
Hideouts	Store hunters, stamina recovery, share info	Completed
Simulation Management	Game loop, entity updates, end conditions	Completed
Visual Rendering	2D grid with scalable resolution	Completed
Sound Feedback	Footsteps, treasure collection, alerts	Completed

2 Modules / Functionality

The Knights of Eldoria simulation consists of multiple functional components, each responsible for a different aspect of the game's execution. These components work together to maintain a logical, structured simulation of AI-based movement, interactions, and event handling.

The primary focus of this section is to break down these modules and explain their function within the overall simulation. This includes an overview of AI decision-making, entity interactions, and rendering mechanisms. By understanding these components, one can appreciate the underlying game mechanics and system architecture that allow for dynamic AI-driven simulation.

2.1 Process Logic (AI & Gameplay)

At the heart of Knights of Eldoria lies a robust AI system that simulates the behaviours of treasure hunters and knights. Treasure hunters navigate the grid using the A pathfinding algorithm, which allows them to find the shortest path to a destination while avoiding obstacles. Their primary objectives include locating the most valuable treasures, storing them in nearby hideouts, and maintaining their stamina.

Hunters exhibit decision-making autonomy. They avoid knights when detected within a danger zone and change their target treasure based on proximity and value. Stamina serves as a critical constraint; hunters lose energy with each move and must retreat to a hideout when stamina falls below a threshold. They also share discovered locations of treasures and threats with fellow hunters when resting at the same hideout.

Knights, functioning as patrolling AI agents, detect hunters within a three-cell radius and pursue them using calculated paths. They expend energy during chases and must recover at garrisons when depleted. When engaging a hunter, a knight may detain or challenge them, resulting in stamina loss for the hunter and possible loss of treasure. This combination of action, memory, and strategy creates a dynamic simulation where each step alters the game's landscape.

2.2 Visual Implementation (Pygame 2D Grid)

The visual layer of Knights of Eldoria serves a dual purpose: to provide clear real-time feedback on game states and to create an engaging user interface that makes the simulation accessible and readable. Implemented using the Pygame library, the game's 2D grid renders a vibrant, modular representation of the world where every cell is uniquely tied to gameplay logic.

Each cell on the grid is capable of housing one or more entities. These include treasure hunters, knights, treasure items, and hideouts. Entities are drawn using custom-designed or open-license sprites that communicate identity and action at a glance. For instance, treasure items are distinguished by colour and iconography based on type—bronze, silver, and gold—while knights are rendered in darker tones to suggest danger.

Real-time rendering is achieved through frame-by-frame updates, ensuring that movements and interactions are immediately visible to observers. The system supports scalable resolution, meaning

the size of each grid cell adjusts based on the display window. When a hunter moves from one cell to another, the change is visualized as a sprite transition. If the hunter is carrying treasure, a visual indicator reflects this change in state.

Hideouts are represented by forest-themed icons, positioned strategically across the grid. When hunters enter a hideout, their sprite is either shaded or replaced by a resting icon, depending on the configuration. Knights patrolling the grid exhibit more aggressive movement animations, making it easier to distinguish pursuit versus idle behaviour.

A significant part of the visual design also includes grid overlays and UI elements. Each cell is outlined, allowing viewers to track movement. A statistics overlay displays active hunters, collected treasure count, knight energy levels, and hunter stamina bars. These overlays are refreshed in real time, giving the simulation a clear and interactive feel despite its autonomous nature.

To complement visuals, sound effects are incorporated to enhance immersion. Footsteps accompany movements, a distinct sound plays on treasure collection, and a warning sound is triggered when knights engage in a chase. The combination of audio-visual feedback not only aids gameplay observation but also makes the system suitable for demonstration or educational purposes.

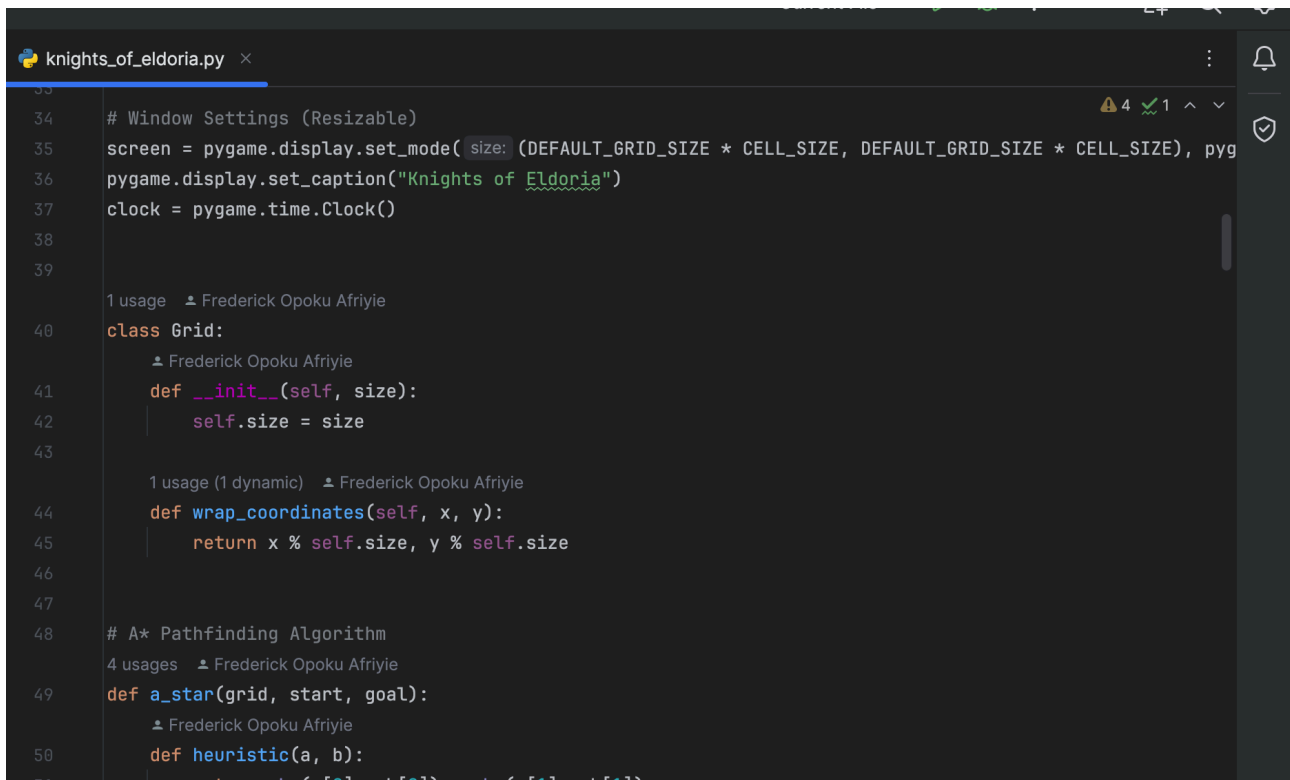
In essence, the visual system bridges the gap between backend AI logic and user comprehension, transforming a complex set of algorithms into an observable, meaningful simulation experience.

2.3 Main Game Module (Code Breakdown)

The game is structured around modular, object-oriented classes that simulate a living ecosystem of treasure hunters, knights, resources, and safe zones. Each module is designed to handle a specific gameplay function and is responsible for its own logic, state management, and interface with the simulation controller.

Grid Class:

The Grid class initializes the game world with dimensions and logic that wrap around horizontally and vertically. This approach mimics an infinite map, allowing agents to reappear from the opposite end when reaching a boundary. The class provides utility methods for positioning, occupancy checking, and rendering coordination.



```
knights_of_eldoria.py x
33
34 # Window Settings (Resizable)
35 screen = pygame.display.set_mode(size: (DEFAULT_GRID_SIZE * CELL_SIZE, DEFAULT_GRID_SIZE * CELL_SIZE), pyg
36 pygame.display.set_caption("Knights of Eldoria")
37 clock = pygame.time.Clock()
38
39
40 class Grid:
41     Frederick Opoku Afriyie
42     def __init__(self, size):
43         self.size = size
44
45     1 usage (1 dynamic) Frederick Opoku Afriyie
46     def wrap_coordinates(self, x, y):
47         return x % self.size, y % self.size
48
49 # A* Pathfinding Algorithm
50 4 usages Frederick Opoku Afriyie
51 def a_star(grid, start, goal):
52     Frederick Opoku Afriyie
53     def heuristic(a, b):
```

Figure 1 Screenshot of Grid Class code

Treasure Class:

Each treasure has a type (bronze, silver, or gold), which determines its value and prioritization level by hunters. It contains a decay mechanism, reducing its value by 0.1% per simulation step. When its value reaches zero, it is removed from the game. This class also ensures treasure cannot be duplicated or reused once collected.

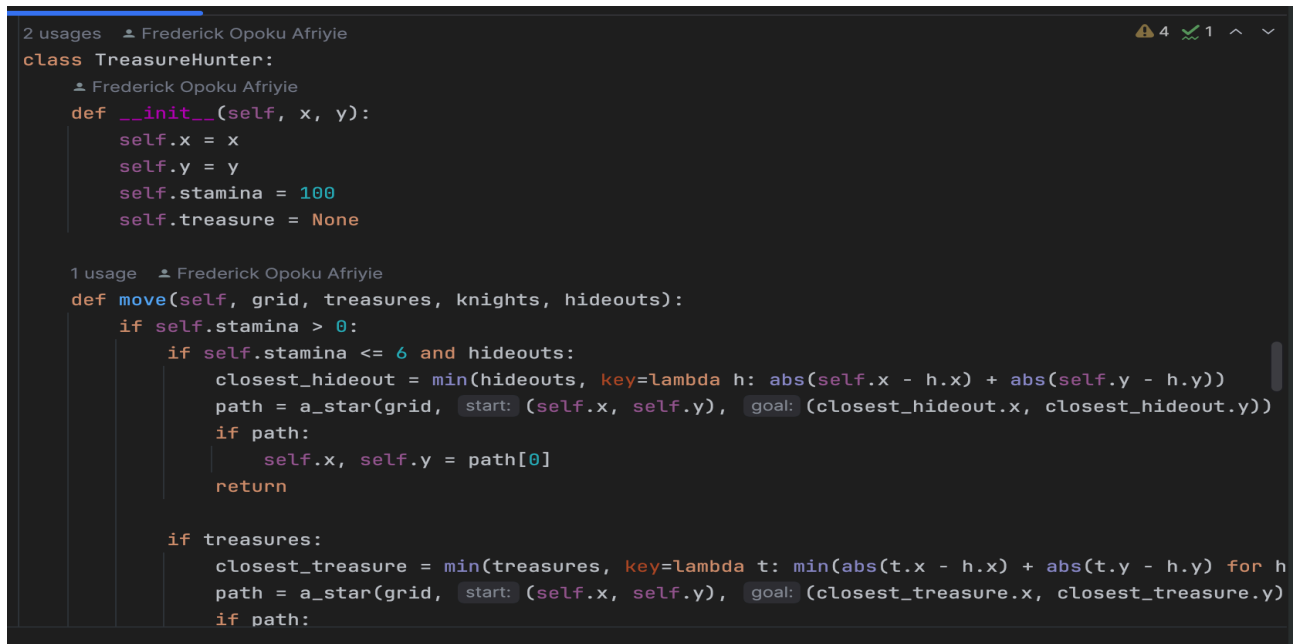


```
1 usage Frederick Opoku Afriyie
87 class Treasure:
88     Frederick Opoku Afriyie
89     def __init__(self, x, y, treasure_type):
90         self.x = x
91         self.y = y
92         self.type = treasure_type
93         self.value = {"bronze": 3, "silver": 7, "gold": 13}[treasure_type]
94
95     1 usage Frederick Opoku Afriyie
96     def decay(self):
97         self.value -= 0.1 * self.value
98         if self.value <= 0:
99             self.value = 0
```

Figure 2 Screenshot of Treasure Class

TreasureHunter Class:

This class embodies hunter behaviour. It includes stamina management, movement, decision-making, and cooperation with other hunters. The use of A* pathfinding allows hunters to navigate efficiently. They also memorize key locations like nearby treasures and hideouts. They can only carry one treasure at a time and automatically evaluate its value against effort, proximity, and knight risk. If threatened or low on stamina, the AI prioritizes retreat to the nearest hideout. Hunters share information during rest, enhancing team behaviour.

A screenshot of a code editor with a dark theme. The editor shows the implementation of the 'TreasureHunter' class. At the top, there are two tabs: '2 usages' and 'Frederick Opoku Afriyie'. The code defines the class 'TreasureHunter' with an '.__init__' method that initializes 'self.x', 'self.y', 'self.stamina' (set to 100), and 'self.treasure' (set to None). Below this, there is a '1 usage' tab and the 'move' method. The 'move' method takes 'self', 'grid', 'treasures', 'knights', and 'hideouts' as arguments. It first checks if 'self.stamina > 0'. If true, it checks if 'self.stamina <= 6 and hideouts'. If this condition is met, it finds the 'closest_hideout' using a lambda function with 'abs' and 'a_star' pathfinding, then updates 'self.x' and 'self.y' to the first element of the path and returns. If the stamina condition is not met, it checks if there are 'treasures'. If true, it finds the 'closest_treasure' using a lambda function with 'abs' and 'a_star' pathfinding, then updates 'self.x' and 'self.y' to the first element of the path. The code is color-coded: keywords in blue, strings in red, and comments in green. The editor interface includes a top bar with icons for warnings (4), errors (1), and other settings.

```
2 usages  Frederick Opoku Afriyie
class TreasureHunter:
    Frederick Opoku Afriyie
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.stamina = 100
        self.treasure = None

1 usage  Frederick Opoku Afriyie
def move(self, grid, treasures, knights, hideouts):
    if self.stamina > 0:
        if self.stamina <= 6 and hideouts:
            closest_hideout = min(hideouts, key=lambda h: abs(self.x - h.x) + abs(self.y - h.y))
            path = a_star(grid, start=(self.x, self.y), goal=(closest_hideout.x, closest_hideout.y))
            if path:
                self.x, self.y = path[0]
            return

        if treasures:
            closest_treasure = min(treasures, key=lambda t: min(abs(t.x - h.x) + abs(t.y - h.y) for h
            path = a_star(grid, start=(self.x, self.y), goal=(closest_treasure.x, closest_treasure.y))
            if path:
```

Figure 3 Screenshot of Treasure Hunter Class

Knight Class:

Knights follow a patrol AI pattern with added memory. When no hunter is in range, they patrol based on previous activity zones. When a hunter is detected (within a 3-cell radius), the knight chases and drains energy. If energy drops below 20%, the knight autonomously retreats to a garrison and recovers. During encounters, knights can either detain (light penalty) or challenge (high penalty) a hunter, forcing treasure drop.

```
2 usages  Frederick Opoku Afriyie
class Knight:
    Frederick Opoku Afriyie
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.energy = 100

1 usage  Frederick Opoku Afriyie
def patrol(self, hunters, grid):
    if hunters:
        closest_hunter = min(hunters, key=lambda h: abs(self.x - h.x) + abs(self.y - h.y))
        path = a_star(grid, start=(self.x, self.y), goal=(closest_hunter.x, closest_hunter.y))
        if path:
            self.x, self.y = path[0]
            self.energy -= 20

        if self.energy <= 20:
            self.energy += 10
```

Figure 4 Screenshot of Knight Class

Hideout Class:

Hideouts are safe zones. Each hideout hosts up to five hunters. Hunters can rest to restore stamina (1% per step) and share memory about environment changes. Hideouts are also recruitment zones; if diverse skills are present, there's a 20% chance a new hunter will be spawned with a randomly assigned skill.

```
2 usages  Frederick Opoku Afriyie
class Hideout:
    Frederick Opoku Afriyie
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.capacity = 5
        self.hunters = []

    Frederick Opoku Afriyie
    def rest_hunters(self):
        for hunter in self.hunters:
            hunter.stamina = min(100, hunter.stamina + 1)
```

Figure 5 Screenshot of Hideout Class

Simulation Class:

This orchestrator runs the entire game loop. It spawns the environment, updates entities each tick, applies decay, renders states, and checks end conditions (no treasure or all hunters gone). The simulation ensures harmony between AI actions and visual updates.

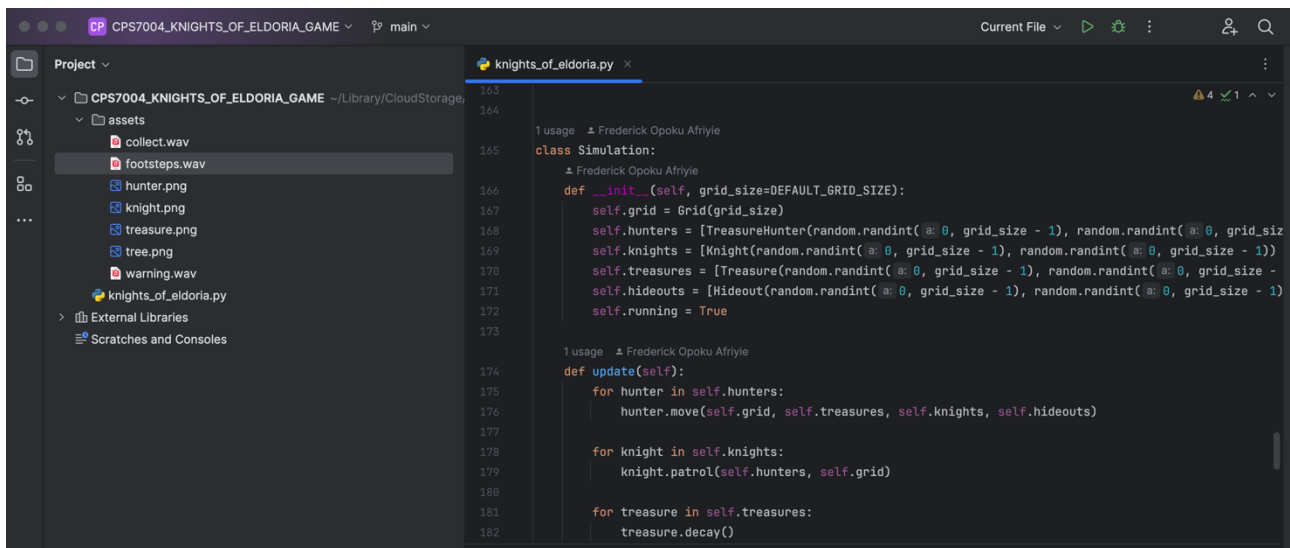


Figure 6 Screenshot of Simulation Class

3 Recommendations

Although the current implementation meets all core requirements for a self-operating AI simulation, there are several areas where the game can be enhanced both technically and experientially.

3.1 Future Enhancements

While Knights of Eldoria meets all the core functional requirements for a strategic, AI-driven simulation, its modular design makes it an ideal candidate for iterative expansion. The following enhancements could elevate the experience, both from a gameplay and technical AI standpoint:

1. Machine Learning Integration:

Incorporating reinforcement learning techniques such as Q-learning could allow hunters and knights to learn from past encounters. This would create a progressively challenging environment and simulate intelligent adaptation.

2. Multiplayer or Player-Controlled Mode:

Introducing the ability for players to control a hunter, while others remain AI-driven, would add strategy and engagement.

3. Advanced Animation and UI:

Replace static icons with sprite-based animations. Add UI overlays showing hunter health, collected

treasure, and real-time notifications.

4. Extended Terrain Types:

Introduce terrain with movement penalties (e.g., swamps, hills) or bonuses (e.g., roads) to diversify pathfinding strategies.

5. Night/Day Cycle:

Environmental changes like time-based visibility would make gameplay more realistic and affect AI decisions.

6. Online Scoreboard:

Track successful runs and display global scores if multiplayer is implemented.

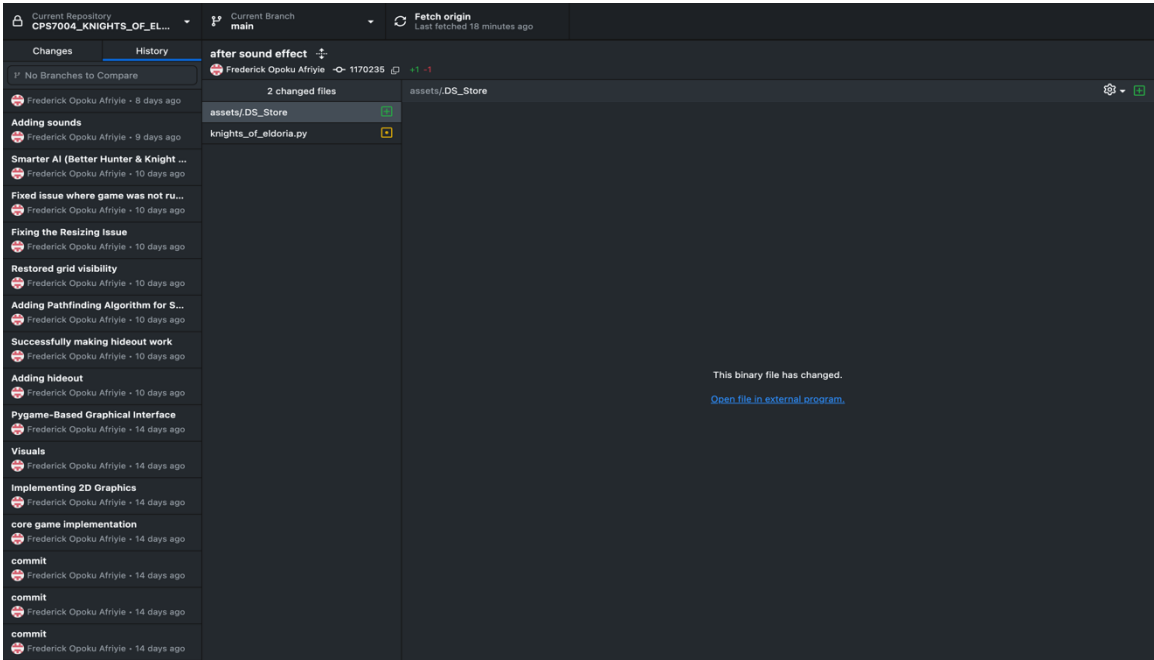
These suggestions aim to increase engagement, strategic depth, and realism while providing a platform for future research in AI behaviours.

4 References

- Sweigart, A. (2012). *Making Games with Python & Pygame*. Houston, Texas.
- McGugan, W. (2007). *Beginning Game Development with Python and Pygame* (Vol. illustrated).
northwest England, United Kingdom: Apress.
- Rusell, S. %. (2020). *Artificial Intelligence: A Modern Approach* (Vol. 4th ed). Pearson.
- Association for computing* . (2018). ACM Code of Ethics and Professional Conduct.
- Alejandro Rodas de Paz, J. H. (2015). *Python Game Programming By Example*. Seville, Spain:
Packt Publishing Ltd,.

5 Appendices

- **Git Commit History** (for version control evidence)



- **Screenshots of the Final Game UI**

